UF2405. UD4. ACTIVIDAD 1
Ejercicio teórico-práctico: Funciones de diferentes códigos

Módulo formativo: UF2405
Unidad didáctica: UD4
Actividad: Funciones de diferentes códigos

INTRODUCCIÓN

El objetivo de esta actividad es identificar y describir las funciones que aparecen en diferentes fragmentos de código PHP relacionados con la conexión a bases de datos relacionales, la consulta de información, la eliminación de registros y la inserción de datos.

Para comprender su funcionamiento, se analizan tres ejemplos de código y se relacionan con un proyecto demostrativo desarrollado en PHP y MySQL/MariaDB. Este proyecto permite poner en práctica las operaciones descritas mediante una aplicación web sencilla.

La actividad se relaciona con el criterio de evaluación CE2.1, que consiste en enumerar y describir las diferentes tecnologías de conexión con bases de datos desde las aplicaciones.

PREGUNTA 1. DESCRIBIR QUÉ HACE EL SIGUIENTE CÓDIGO

1.1. Finalidad del código

El código utiliza PHP para conectarse a un servidor MySQL, seleccionar una base de datos, ejecutar una consulta SQL de selección y mostrar los registros obtenidos en una tabla HTML.

Una vez mostrados los resultados, libera los recursos utilizados por la consulta y cierra la conexión con el servidor de base de datos.

Se trata, por tanto, de un ejemplo de lectura y visualización de información almacenada en una base de datos relacional.

1.2. Funcionamiento paso a paso

1. Conexión con el servidor MySQL

La instrucción mysql_connect() establece una conexión con el servidor de base de datos mediante el nombre del servidor, el usuario y la contraseña.

La variable $link almacena el identificador de la conexión. Si la conexión falla, la función die() detiene la ejecución del programa y muestra un mensaje de error junto con la información proporcionada por mysql_error().

Si la conexión se establece correctamente, se muestra el mensaje «Connected successfully».

2. Selección de la base de datos

La función mysql_select_db() selecciona la base de datos denominada my_database, que será la utilizada para ejecutar las consultas.

Si no se puede seleccionar, el programa se detiene y muestra un mensaje de error.

3. Preparación y ejecución de la consulta SQL

La variable $query contiene la siguiente instrucción:

SELECT * FROM my_table

Esta consulta solicita todos los registros y todas las columnas de la tabla my_table.

La función mysql_query() ejecuta la consulta y almacena el resultado en la variable $result. Si se produce un error, se detiene la ejecución y se muestra el mensaje correspondiente.

4. Creación de la tabla HTML

La instrucción echo genera la etiqueta de apertura de una tabla HTML mediante <table>.

A continuación, el bucle while utiliza mysql_fetch_array() para recuperar los registros uno a uno. El parámetro MYSQL_ASSOC indica que cada registro se devuelve como un array asociativo, en el que los nombres de las columnas se utilizan como claves.

Por cada registro recuperado, se crea una fila mediante la etiqueta <tr>.

El bucle foreach recorre los valores de las columnas del registro y genera una celda <td> para cada valor.

Cuando no quedan más registros, el bucle while finaliza y se escribe la etiqueta de cierre </table>.

5. Liberación de recursos

La función mysql_free_result() libera la memoria asociada al resultado de la consulta.

Finalmente, mysql_close($link) cierra la conexión con el servidor de base de datos.

1.3. Resultado del código

Si la conexión, la selección de la base de datos y la consulta se realizan correctamente, el programa muestra una tabla HTML con los registros de my_table.

Cada fila de la tabla corresponde a un registro de la base de datos, y cada celda contiene el valor de una de sus columnas.

1.4. Relación con el proyecto demostrativo

En el proyecto demostrativo se reproduce esta funcionalidad mediante una conexión con MySQL/MariaDB y una consulta de selección de registros.

La información recuperada se presenta en una tabla HTML, de forma que el usuario puede visualizar los datos almacenados.

La implementación práctica utiliza la extensión mysqli de PHP y métodos como fetch_assoc() para obtener los registros como arrays asociativos.

Esta solución permite comprender el proceso completo de conexión, consulta, recuperación de datos y presentación de resultados.

1.5. Consideraciones técnicas

Las funciones mysql_connect(), mysql_select_db(), mysql_query(), mysql_fetch_array(), mysql_free_result() y mysql_close() pertenecen a la antigua extensión mysql de PHP, que fue eliminada en PHP 7.0.

Por ello, este código no puede ejecutarse directamente en versiones modernas de PHP sin adaptarlo.

Actualmente, se recomienda utilizar mysqli o PDO para trabajar con bases de datos MySQL. Asimismo, los datos que se muestran en HTML deben escaparse adecuadamente para evitar que un contenido almacenado en la base de datos pueda interpretarse como código HTML o JavaScript.

PREGUNTA 2. ¿QUÉ FUNCIÓN REALIZA EL SIGUIENTE CÓDIGO?

2.1. Finalidad del código

El código permite eliminar una empresa de la tabla empresa de una base de datos cuando se recibe una petición con la acción del.

Para ello, carga los archivos de conexión y funciones auxiliares, obtiene los parámetros enviados mediante la URL, construye una consulta SQL de eliminación, la ejecuta y redirige al usuario a la página del listado.

Se trata de una operación de eliminación de registros, conocida como DELETE en SQL.

2.2. Funcionamiento paso a paso

1. Inclusión de los archivos necesarios

Las instrucciones:

require("conexion.php");

require("funciones.php");

incluyen los archivos que contienen, respectivamente, la configuración o conexión con la base de datos y las funciones auxiliares utilizadas por el programa.

La instrucción require permite reutilizar código común en diferentes páginas de una aplicación. Si el archivo requerido no puede cargarse, normalmente se produce un error que impide continuar la ejecución.

2. Obtención del identificador de la empresa

La instrucción:

$idempresa = getParam($_GET["id"], "-1");

obtiene el parámetro id enviado mediante el método GET, habitualmente a través de la URL.

La función auxiliar getParam() parece utilizarse para recuperar o normalizar el parámetro y establecer -1 como valor predeterminado si no se proporciona un valor válido. El comportamiento exacto depende de cómo esté implementada esta función en funciones.php.

El identificador permite determinar qué empresa se desea eliminar.

3. Obtención de la acción solicitada

La instrucción:

$action = getParam($_GET["action"], "");

recupera el parámetro action de la URL.

Si el parámetro no se proporciona, se utiliza una cadena vacía como valor predeterminado.

4. Comprobación de la acción

La condición:

if ($action == "del")

comprueba si la acción solicitada es del.

Solo cuando se cumple esta condición se ejecuta el bloque de instrucciones encargado de eliminar el registro.

5. Construcción de la consulta SQL

La instrucción:

$sql = "DELETE FROM empresa WHERE id = ".sqlValue($idempresa, "int");

construye una consulta SQL que elimina el registro de la tabla empresa cuyo campo id coincide con el identificador recibido.

La cláusula DELETE FROM empresa indica la tabla en la que se eliminarán los registros, mientras que WHERE id = ... especifica qué registro se desea borrar.

La función auxiliar sqlValue() parece convertir o preparar el identificador como un valor de tipo entero. Su comportamiento exacto depende de la implementación incluida en el archivo funciones.php.

La cláusula WHERE es esencial: sin ella, la consulta podría eliminar todos los registros de la tabla.

6. Ejecución de la eliminación

La instrucción:

mysql_query($sql, $conexion);

ejecuta la consulta SQL utilizando la conexión almacenada en $conexion.

Si la consulta se ejecuta correctamente, el registro que cumple la condición deja de estar disponible en la tabla, salvo que existan restricciones u otras condiciones que impidan la operación.

El código original no comprueba explícitamente el resultado de esta ejecución, por lo que no ofrece un tratamiento específico de los posibles errores.

7. Redirección al listado

La instrucción:

header("location: listado.php");

envía una cabecera HTTP para redirigir el navegador a la página listado.php.

De esta manera, el usuario vuelve al listado de empresas después de solicitar la eliminación. Para que la redirección funcione correctamente, no debe haberse enviado previamente contenido al navegador.

2.3. Resultado del código

Cuando se recibe la acción del, el programa intenta eliminar de la tabla empresa el registro cuyo identificador coincide con el parámetro id.

Después, redirige al usuario a listado.php.

Si la acción no es del, no se ejecuta el bloque de eliminación.

2.4. Relación con el proyecto demostrativo

En el proyecto demostrativo se implementa una operación equivalente mediante una página PHP dedicada a la eliminación de empresas.

La aplicación recupera el identificador del registro, realiza las comprobaciones necesarias, ejecuta una consulta SQL de tipo DELETE y permite volver al listado.

Esta práctica permite comprender cómo se conectan las operaciones de una aplicación web con las tablas de una base de datos relacional.

También muestra la importancia de separar la conexión, las funciones auxiliares y la lógica de cada operación en archivos diferentes.

2.5. Consideraciones técnicas y de seguridad

La extensión antigua mysql ya no está disponible en PHP 7.0 ni en versiones posteriores. Por ello, en una aplicación moderna debe sustituirse por mysqli o PDO.

Además, una aplicación real debe comprobar que el usuario tiene permisos para eliminar empresas. La acción de borrado no debería ejecutarse únicamente por acceder a una URL.

Es recomendable utilizar una consulta preparada para evitar la construcción insegura de consultas SQL, validar el identificador, comprobar los errores de ejecución y proteger las operaciones que modifican datos frente a solicitudes no autorizadas.

El uso de una consulta preparada no sustituye a la comprobación de permisos ni a las medidas de protección frente a peticiones falsificadas.

PREGUNTA 3. ¿QUÉ FUNCIÓN REALIZA EL SIGUIENTE CÓDIGO?

3.1. Finalidad del código

El código establece una conexión con un servidor MySQL, selecciona una base de datos, configura la codificación de caracteres en UTF-8 e inserta dos registros en la tabla agenda.

Una vez realizadas las operaciones, cierra la conexión y muestra un mensaje indicando que los datos han sido insertados.

Se trata de un ejemplo de escritura de información en una base de datos relacional mediante sentencias SQL de tipo INSERT.

3.2. Funcionamiento paso a paso

1. Conexión con el servidor MySQL

La instrucción:

$link = mysqli_connect("sql203.byethost7.com", "b7_10356956", "*****");

utiliza mysqli_connect() para establecer una conexión con el servidor de base de datos.

Los parámetros corresponden al nombre del servidor, el usuario y la contraseña. En el ejemplo, la contraseña aparece sustituida por asteriscos.

El resultado de la conexión se almacena en la variable $link.

2. Selección de la base de datos

La instrucción:

mysqli_select_db($link, "b7_10356956_mibasededatos");

selecciona la base de datos con la que se trabajará durante la ejecución del programa.

El primer parámetro identifica la conexión y el segundo indica el nombre de la base de datos.

3. Configuración de la codificación de caracteres

La instrucción:

$tildes = $link->query("SET NAMES 'utf8'");

ejecuta una instrucción SQL para configurar la codificación de caracteres de la conexión.

Su finalidad es facilitar el tratamiento correcto de caracteres especiales, como las vocales acentuadas y la letra ñ, presentes en nombres y apellidos.

En las aplicaciones modernas desarrolladas con MySQL/MariaDB suele recomendarse configurar explícitamente la conexión con mysqli_set_charset($link, "utf8mb4"), siempre que la base de datos utilice una codificación compatible.

4. Inserción del primer registro

La instrucción:

mysqli_query($link, "INSERT INTO agenda VALUES ('Pedro2', 'Gómez Gómez', 'C/ Buenaventura 54', '699887766', 35, 1.98)");

ejecuta una consulta SQL que inserta un nuevo registro en la tabla agenda.

Los valores representan, aparentemente, el nombre, los apellidos, la dirección, el teléfono, la edad y la altura de una persona.

El orden y el significado exacto de los campos dependen de la estructura definida para la tabla agenda.

5. Inserción del segundo registro

La siguiente instrucción realiza una segunda operación INSERT:

mysqli_query($link, "INSERT INTO agenda VALUES ('Antonio2', 'Delgado Folken', 'C/ El Milagroso 77', '699667744', 88, 1.71)");

En este caso, se añade otro registro con los datos de una persona diferente.

Al igual que en la inserción anterior, el número y el orden de los valores deben coincidir con las columnas de la tabla.

6. Cierre de la conexión

La instrucción:

mysqli_close($link);

cierra la conexión con el servidor de base de datos una vez finalizadas las operaciones.

Cerrar la conexión ayuda a liberar los recursos utilizados por la aplicación.

7. Mensaje de confirmación

Finalmente, la instrucción:

echo 'Los datos han sido insertados en la base de datos';

muestra un mensaje informativo en el navegador.

No obstante, en el código original este mensaje se muestra sin comprobar previamente si las dos inserciones se han realizado correctamente. Por tanto, el mensaje no garantiza por sí solo que los registros se hayan guardado.

3.3. Resultado del código

Si la conexión es correcta, la base de datos está disponible, la tabla existe y las dos consultas INSERT se ejecutan correctamente, se añaden dos registros a la tabla agenda.

Posteriormente, se cierra la conexión y se muestra el mensaje de confirmación.

3.4. Relación con el proyecto demostrativo

En el proyecto demostrativo se implementa una operación equivalente mediante un formulario de entrada de datos y una página PHP encargada de procesarlos.

El usuario introduce la información de los contactos y la aplicación valida los campos antes de guardar los registros en la base de datos.

La implementación utiliza mysqli y consultas preparadas para realizar la inserción de los datos.

Este procedimiento permite comprender el recorrido de la información desde el formulario HTML hasta la base de datos: recepción de los datos, validación, ejecución de la consulta SQL y presentación del resultado.

3.5. Consideraciones técnicas y de seguridad

El código utiliza mysqli, una extensión de PHP que permite trabajar con bases de datos MySQL y MariaDB. A diferencia de la antigua extensión mysql, mysqli continúa disponible en las versiones modernas de PHP.

En una aplicación real es recomendable comprobar el resultado de la conexión, la selección de la base de datos y cada consulta ejecutada.

También es preferible indicar explícitamente las columnas de destino en la sentencia INSERT, en lugar de depender del orden de todas las columnas de la tabla.

Por seguridad, se recomienda utilizar consultas preparadas, validar los datos recibidos del usuario y proteger las credenciales de conexión. Además, el mensaje de confirmación debe mostrarse únicamente cuando las operaciones se hayan completado correctamente.

CONCLUSIÓN GENERAL

Los tres fragmentos de código muestran operaciones fundamentales para conectar una aplicación web con una base de datos relacional.

El primer ejemplo permite consultar información mediante SELECT y mostrar los resultados en una tabla HTML. El segundo utiliza DELETE para eliminar un registro concreto de la tabla empresa. El tercero utiliza INSERT para añadir nuevos registros a la tabla agenda.

A través del proyecto demostrativo se comprueba cómo estas operaciones pueden implementarse con PHP y MySQL/MariaDB utilizando la extensión mysqli.

El desarrollo permite identificar las funciones de conexión, selección de bases de datos, ejecución de consultas SQL, recuperación de resultados y cierre de conexiones. También pone de manifiesto la importancia de validar los datos, comprobar los errores y aplicar medidas de seguridad.

En relación con el criterio CE2.1, los ejemplos permiten distinguir entre la antigua extensión mysql, actualmente obsoleta y eliminada de PHP, y las tecnologías de conexión disponibles en versiones modernas, como mysqli y PDO.

Por tanto, la actividad contribuye a comprender cómo las aplicaciones acceden a la información almacenada en bases de datos relacionales y cómo realizan operaciones de lectura, inserción y eliminación de registros de forma estructurada.